



Linux e la shell Bash

Stream ed espressioni regolari

Sistemi Operativi

Laurea Triennale in Ingegneria Informatica
e dell'Automazione, A.A. 2024/2025

Indice

- Stream e redirectione
- Pipeline di comandi
 - echo, cat, more, less, sort, head, tail, uniq, wc, cut, diff
- Esercizi
 - Soluzioni
- Espressioni regolari
 - grep, find, sed

Stream e redirectione (1/2)

- Ciascun programma dispone di tre **stream standard**, identificati dai **file descriptor** 0, 1 e 2:
 - 0 - **Standard input**, *stdin* (default: tastiera)
 - 1 - **Standard output**, *stdout* (default: terminale)
 - 2 - **Standard error**, *stderr* (default: terminale)
- Per effettuare la **redirezione**:
 - *comando* [**file_descriptor**] <**operatore**> *file*
 - Se non si specifica il file descriptor, ogni operatore assume che ci si riferisca ad uno degli stream standard.
- **Operatori**:
 - < : redirezione input (default: *stdin*).
 - > : redirezione output in modalità truncate (default: *stdout*).
 - >> : redirezione output in modalità append (default: *stdout*).
 - >& : concatenazione di due file descriptor (default: *stdout*).

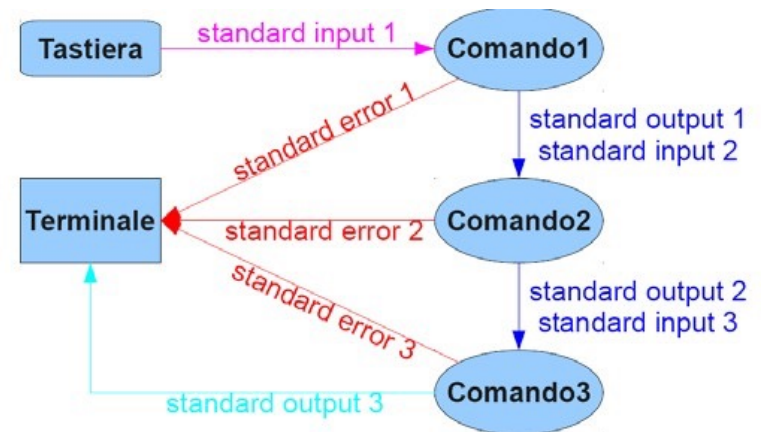
Stream e redirectione (2/2)

○ Esempi:

- `cmd > myfile.txt` : redirige *stdout* in *myfile.txt* (truncate mode).
- `cmd 2>> myfile.txt` : redirige *stderr* in *myfile.txt* (append mode).
- `cmd > myfile.txt 2>&1` : redirige *stdout* in *myfile.txt*, e concatena *stderr* a *stdout*. *stdout* e *stderr* finiscono **entrambi** in *myfile.txt*.
Equivalente a: `cmd &> myfile.txt`
- `cmd < myin.txt > myout.txt 2> myerr.txt` : redirige *stdin*, *stdout* e *stderr* in *myin.txt*, *myout.txt* e *myerr.txt*, rispettivamente.
- `cmd < myin.txt >> myout.txt 2>&1` : redirige *stdin* in *myin.txt*, *stdout* in *myout.txt* (append mode), e concatena *stderr* a *stdout* (entrambi finiscono in *myout.txt*, in append mode).

Sequenze di comandi

- È possibile eseguire **più comandi in una stessa riga**:
 - **comando1 ; comando2** : esegue *comando1* e *comando2* in sequenza.
 - **comando1 && comando2** : esegue *comando1*; se questo ha successo (exit code 0) esegue anche *comando2*. Simile ad AND booleano.
 - **comando1 || comando2** : esegue *comando1*; se questo fallisce, esegue *comando2*. Simile ad OR booleano. - -
- Una sequenza di comandi particolarmente utile si ottiene utilizzando il carattere di pipe "|":
 - **comando1 | comando2** : esegue *comando1*, ne redirige *stdout* in *stdin* di *comando2*, poi esegue *comando2*.
- Ciò consente di creare **pipeline** con cui elaborare sequenzialmente l'output di qualunque comando.



echo

- Stampa un messaggio su standard output.
- **Sintassi:**
 - `echo [stringa]`
- **Esempi:**
 - `echo "Hello, world!"` : stampa il messaggio specificato su *stdout*.
 - `echo ~` : stampa il percorso della directory utente su *stdout*.
 - `echo "Hello, world!" > myfile.txt` : stampa il messaggio specificato in *myfile.txt*, modalità truncate.
 - `echo "Hello, world!" >> myfile.txt` : stampa il messaggio specificato in *myfile.txt*, modalità append.

cat

- Concatenate.
- Concatena il contenuto di più file.
- **Sintassi:**
 - `cat [flags] [percorso] ...`
- **Flag:**
 - `-n` : Numera le righe.
- **Esempi:**
 - `cat myfile1.txt myfile2.txt` : concatena il contenuto di *myfile1.txt* e *myfile2.txt* e lo stampa su *stdout*.
 - `cat myfile1.txt` : stampa il contenuto di *myfile.txt* su *stdout*.
 - `cat myfile1.txt myfile2.txt > myfile3.txt` : concatena il contenuto di *myfile1.txt* e *myfile2.txt* e lo inserisce in *myfile3.txt*.

more, less

- Programmi di **paginazione**, visualizzano file in maniera interattiva.
- **More** è stato rimpiazzato da **less**, più diffuso e completo. Quest'ultimo consente di scrollare all'indietro (invece che solo in avanti, come nelle prime versioni di *more*) e fornisce funzionalità avanzate di ricerca.
- **Sintassi:**
 - `more <percorso>`
 - `less <percorso>`
- **Esempi:**
 - `less myfile.txt` : visualizza *myfile.txt* nel paginatore.
 - `ls -la | less` : visualizza l'output del comando specificato nel paginatore.

sort

- **Ordina** le righe di uno o più file.
 - Di default per **ordine lessicografico**.
- **Sintassi:**
 - `sort [flags] <percorso> ...`
- **Flag:**
 - **-c** : verifica che il file sia ordinato.
Se non lo è, stampa la prima riga che non rispetta l'ordine.
 - **-n** : ordina secondo il valore numerico.
 - **-f** : ordinamento case-insensitive.
 - **-r** : inverte l'ordinamento.
 - **-k N** : ordina secondo il campo N-esimo. Il separatore di default è lo spazio, ma può essere controllato con **-t**.

head

- Mostra le prime righe o byte di un file.
 - Di default mostra le prime 10 righe.
- **Sintassi:**
 - `head [flags] <percorso>`
- **Flag:**
 - `-c N` : mostra i primi N byte.
 - `-c -N` : mostra i primi byte, esclusi gli ultimi N.
 - `-n N` : mostra le prime N righe (equivalente a `-N`).
 - `-n -N` : mostra le prime righe, escluse le ultime N.

tail

- Mostra le ultime righe o byte di un file.
 - Di default mostra le ultime 10 righe.
- **Sintassi:**
 - `tail [flags] <percorso>`
- **Flag:**
 - `-c N`: mostra gli ultimi N byte.
 - `-c +N`: mostra i byte a partire da N.
 - `-n N`: mostra le ultime N righe (equivalente a `-N`).
 - `-n +N`: mostra le righe a partire da N.

uniq

- Mostra o omette le righe ripetute.
 - Di default mostra tutte le righe, omettendo le ripetizioni.
 - **Nota:** richiede che le righe ripetute siano adiacenti.
 - Un modo semplice per rispettare questo vincolo è ordinare con **sort**.
- **Sintassi:**
 - `uniq [flags] <percorso>`
- **Flag:**
 - **-d** : mostra solo le linee duplicate.
 - **-u** : mostra solo le linee che non hanno duplicati.
 - **-c** : indica il numero di occorrenze per ciascuna riga.
 - **-i** : confronto case-insensitive.

- **W**ord **c**ount.
- Conta righe, parole, caratteri e byte per ciascun file.
 - Di default conta righe, parole e byte.
- **Sintassi:**
 - `wc [flags] <percorso>`
- **Flag:**
 - `-l` : conta le righe.
 - `-w` : conta le parole.
 - `-m` : conta i caratteri.
 - `-c` : conta i byte.
- **Output:**
 - `[righe] [parole] [caratteri] [byte] percorso`

cut

- Estrae parti da ciascuna riga di testo.
 - Può estrarre specifici byte, caratteri, o campi delimitati da un carattere separatore.
- **Sintassi:**
 - `cut [flags] <percorso>`
- **Flag:**
 - `-b <range>` : estrae solo i byte indicati nel range.
 - `-c <range>` : estrae solo i caratteri indicati nel range.
 - `-f <range>` : estrae solo i campi indicati nel range.
 - `-d <separatore>` : specifica il separatore dei campi (default: *TAB*).
- Un **range** può avere una delle seguenti forme:
 - `N` : estrae l'elemento N.
 - `N-M` : estrae gli elementi da N ad M.
 - `N-` : estrae gli elementi a partire da N.
 - `-N` : estrae gli elementi fino ad N.

diff

- Trova **diff**erenze tra due file o directory.
 - Utile per confrontare file di configurazione.
 - La variante **colordiff** fornisce output colorato.
- **Sintassi:**
 - `diff [flags] <percorso_1> <percorso_2>`
- **Flag:**
 - **-r** : nel confronto tra due directory, confronta sottodirectory ricorsivamente.
 - **--brief** : riferisce solo se ci sono differenze, senza mostrarle.
 - **--ignore-case** : confronto case-insensitive.

Esercizi

1. Dato il file *elenco.txt*, scrivere una riga di comando atta a visualizzare:
 - La terza e la quarta riga.
 - Le penultime tre righe.
 - L'*N*-esima riga.
2. Scrivere la pipeline di comandi atta a visualizzare il nome dell'ultimo file nella directory di lavoro in ordine lessicografico.
3. Dato il file *elenco.txt*, scrivere una riga di comando atta a creare un file *nome.txt*, contenente la terza parola della seconda riga del file in ordine lessicografico.
4. Scrivere una riga di comando atta a rimuovere la prima riga dal file *elenco.txt*, salvandone il contenuto.

Soluzioni

1. Comandi:

- `head -4 elenco.txt | tail -2`
- `tail -4 elenco.txt | head -3`
- `head -N elenco.txt | tail -1`

2. Soluzione:

- `ls | sort | tail -1`

3. Soluzione:

- `sort elenco.txt | head -2 | tail -1 | cut -f 3 -d " " > nome.txt`

4. Soluzione:

- `tail -n +2 elenco.txt > temp.txt && mv temp.txt elenco.txt`

Espressioni regolari (1/3)

- Un'**espressione regolare** (*regular expression, regex*) è un **pattern** che identifica un insieme di stringhe.
- **Sintassi:**
 - **a** : una singola occorrenza del carattere specificato.
 - **[abcd]** : un singolo carattere tra quelli in parentesi. È anche possibile utilizzare range di caratteri (es: [a-d])
 - **[^abcd]** : un singolo carattere non contenuto tra quelli in parentesi.
 - **.** : una singola occorrenza di un carattere qualunque.
 - **a*** : zero o più occorrenze del carattere specificato.
 - **a+** : una o più occorrenze del carattere specificato.
 - **a?** : al massimo una occorrenza del carattere specificato.
 - **a{N}** : esattamente N occorrenze del carattere specificato.
 - **a{N,}** : almeno N occorrenze del carattere specificato.
 - **a{,N}** : al più N occorrenze del carattere specificato.
 - **a{N, M}** : tra N ed M occorrenze del carattere specificato.

Espressioni regolari (2/3)

○ Sintassi:

- `^a` : cerca il carattere ad inizio della riga.
- `a$` : cerca il carattere a fine riga.
- `\<a` : cerca il carattere ad inizio parola.
- `a\>` : cerca il carattere a fine parola.
- `regex1 | regex2` : OR logico tra regex, match con una o con l'altra.

○ Nella maggior parte dei casi visti in precedenza, è possibile sostituire al singolo carattere un **gruppo** contenente un'espressione regolare ed ottenere risultati che ci si aspetta:

- `(regex)*` : zero o più occorrenze dell'espressione regolare.
- `(regex)+` : una o più occorrenze dell'espressione regolare.
- `(regex)?` : al più un'occorrenza dell'espressione regolare.
- `(regex){N, M}` : tra N ed M occorrenze dell'espressione regolare.
- ...

Espressioni regolari (3/3)

○ Classi di caratteri:

- `[upper:]` : Lettere maiuscole.
- `[lower:]` : Lettere minuscole.
- `[digit:]` : Cifre numeriche.
- `[alpha:]` : Caratteri alfabetici.
- `[alnum:]` : Caratteri alfanumerici.
- `[space:]` : Ogni tipologia di spazio.
- `[blank:]` : Spazi e tabulazioni.
- `[punct:]` : Segni di punteggiatura.

- **Nota:** si utilizzano inserendole nel contesto del selettore di classe di caratteri `[ ]`, es: `[upper:]`

grep (1/2)

- Global **r**egular **e**xpression **p**rint.
- Mostra le righe che corrispondono ad una regex.
- **Sintassi:**
 - `grep [flags] pattern <percorso>`
- **Flag:**
 - **-E** : espressione regolare estesa. Permette di utilizzare i caratteri `?, +, {, }, |, (, e )` senza bisogno di farli precedere da `\`, cosa che invece è necessaria con le espressioni regolari elementari (**-G**, default).
 - **-c** : mostra il numero di righe che corrispondono al pattern.
 - **-n** : aggiunge il numero di riga a ciascun match.
 - **-v** : mostra le righe che non corrispondono al pattern.
 - **-i** : confronto case-insensitive.

grep (2/2)

- **file.txt:**
 - 15 fifteen
 - 14 fourteen
 - 13 thirteen
 - 12 twelve
 - 11 eleven
 - 10 ten
 - 9 nine
 - 8 eight
 - 7 seven
 - 6 six
 - 5 five
 - 4 four
 - 3 three
 - 2 two
 - 1 one
- **grep '[24]' file.txt**
 - 14 fourteen
 - 12 twelve
 - 4 four
 - 2 two
- **grep '^1.*e\$' file.txt**
 - 12 twelve
 - 1 one
- **grep '\<f[a-z]+n\>' file.txt**
 - 15 fifteen
 - 14 fourteen
- **grep 'ee' file.txt**
 - 15 fifteen
 - 14 fourteen
 - 13 thirteen
 - 3 three
- **grep 'e{2\}' file.txt**
 - 15 fifteen
 - 14 fourteen
 - 13 thirteen
 - 3 three
- **grep -E 'e{2}' file.txt**
 - 15 fifteen
 - 14 fourteen
 - 13 thirteen
 - 3 three
- **grep -E '^1[04-9].*te+n\$' file.txt**
 - 15 fifteen
 - 14 fourteen
 - 10 ten

find

- Cerca file nella directory specificata, **ricorsivamente**.
 - Consente di usare **pattern semplici** o **regex**. Nel caso di pattern semplici, gli unici metacaratteri validi sono *, ?, e [abc].
- **Sintassi:**
 - **find** <percorso> [flags]
- **Flag:**
 - **-name** <pattern> : cerca file aventi nome corrispondente al pattern.
 - **-regex** <regex> : cerca file aventi percorso corrispondente alla regex specificata. Per abilitare regex estese: **-regextype posix-extended**
 - **-user, -group** <nome> : filtra in base ad utente/gruppo proprietario.
 - **-type** <tipo> : cerca file del tipo specificato (f per file normali, d per directory, l per soft link)
 - **-mindepth, -maxdepth** <N> : profondità min/max di ricerca nelle dir.
 - **-size** [+ -]N[bckMG] : cerca file con dimensioni pari ad esattamente, più di, o meno di N blocchi (da 512 B), B, KB, MB, GB.

sed

- Stream editor.
- Esegue trasformazioni testuali su stream.
- Sintassi:
 - `sed [flags] <percorso>`
- Flag:
 - `-e <script>` : esegue lo script specificato, in sintassi propria del tool.
- Esempi:
 - `sed -e '/regex/d' file.txt`: elimina (d) le righe corrispondenti alla regex.
 - `sed -e 'N,Md' file.txt`: elimina (d) le righe da N ad M incluso.
 - `sed -e '/regex/!d' file.txt`: elimina (d) le righe senza match (!).
 - `sed -e 's/regex/sostituzione/' file.txt`: sostituisce (s) il primo match della regex specificata per ciascuna riga.
 - `sed -e 's/regex/sostituzione/g' file.txt`: sostituisce tutti i match (g).